Guía de defensa del laboratorio

Programación 1, Laboratorio 2026, grupo 5. Cómo explicar, justificar y demostrar el comparador de opciones de IA

6 de noviembre de 2026

Índice

Cómo leer esta guía **2**

1. Qué pide la letra **3**
- 1.1. El problema en un párrafo . . . 3
- 1.2. Las ocho funcionalidades . . . 3
- 1.3. Las clases obligatorias . . . 3
- 1.4. El menú . . . 3
- 1.5. Las reglas que más se preguntan . . . 3

2. Mapa del programa **4**
- 2.1. Archivos y responsabilidades . . . 4
- 2.2. Quién llama a quién . . . 5
- 2.3. Por qué `main.py` habla y `entities/` no . . . 5
- 2.4. Una operación de punta a punta: comparar . . . 5
- 2.5. El diagrama de la letra frente al código . . . 6
- 2.6. Datos de ejemplo . . . 7

3. Conceptos de POO con el código real **7**
- 3.1. Clase abstracta: `OpcionIA` . . . 7
- 3.2. Herencia: `ServicioAPI` y `PlanMensual` . . . 8
- 3.3. Polimorfismo: `costo_estimado()` . . . 8
- 3.4. Encapsulamiento: atributos privados, `@property` y el único setter . . . 9
- 3.5. Excepción propia: `ErrorLaboratorio` . . . 10
- 3.6. Diccionario de puntajes . . . 10
- 3.7. Referencias a objetos en `Evaluacion` . . . 10
- 3.8. `datetime` . . . 11
- 3.9. `match/case` . . . 11
- 3.10. Contadores de códigos y por qué no `len(lista) + 1` . . . 11

4. Recorrido clase por clase **12**
- 4.1. `Criterio` y `CasoUso` . . . 12
- 4.2. `OpcionIA`, `ServicioAPI` y `PlanMensual` . . . 12
- 4.3. `Evaluacion` . . . 12
- 4.4. `Sistema` . . . 13

5. `main.py`: menús y lectura de datos **16**
5.1. Los menús . . . 16
5.2. Las funciones de lectura . . . 16
5.3. Precondiciones antes de pedir datos . . . 17
5.4. Reparto de la validación entre consola y dominio . . . 17
5.5. Qué pasa con una entrada inválida . . . 18
5.6. Listados . . . 18

6. Reglas de negocio con números verificados **19**
6.1. Costo según el tipo . . . 19
6.2. Puntaje ponderado . . . 19
6.3. Orden de los estados . . . 19
6.4. Igualdad en los límites y el `round(..., 6)` . . . 20
6.5. Comparar reemplaza y los códigos no se reutilizan . . . 20
6.6. Actualizar no cambia códigos ni fechas . . . 20
6.7. Ranking y desempates . . . 21

7. Decisiones de diseño y su porqué **21**

8. Demostración en vivo **22**
8.1. Preparación . . . 22
8.2. Parte A: recorrido completo . . . 23
8.3. Parte B: casos borde . . . 24
8.4. Pedidos improvisados . . . 24

9. Preguntas probables del profesor **25**
9.1. Conceptos . . . 25
9.2. Diseño . . . 26
9.3. Reglas de la letra . . . 26
9.4. ¿Qué pasa si…? . . . 27

10. Errores que no hay que cometer y glosario **27**
10.1. Afirmaciones falsas frecuentes . . . 27
10.2. Glosario . . . 28

Cómo leer esta guía

Esta guía prepara para la auditoría oral del laboratorio. Al terminarla, el lector debe poder explicar cada clase y cada regla, justificar cada decisión y demostrar el programa en vivo.

Convenciones:

- **«Letra, pág. N, sección»** remite a la página y al título de sección del PDF oficial de la letra. Lo que «pide el profesor» sale solo de ahí.
- `archivo`**, líneas N a M** remite al código de la carpeta `laboratorio_2026_grupo5/`. Todo fragmento de código está copiado de esos archivos, sin cambios.
- Todos los números de esta guía se obtuvieron ejecutando el programa (Python 3.14.3) o calculando con la fórmula de la letra.

Si hay poco tiempo, conviene leer en este orden: capítulo 1 (qué pide la letra), secciones 2.4 y 2.5 (una operación completa y el diagrama), capítulo 3 (POO con el código), capítulo 6 (reglas con números),

capítulo 8 (demostración) y capítulo 9 (preguntas).

1. Qué pide la letra

1.1. El problema en un párrafo

Una empresa quiere comparar servicios de IA. El programa, de consola y en Python, registra opciones como Claude, OpenAI o Gemini y recomienda alternativas para distintos casos de uso; no consulta servicios reales. Cada opción tiene un puntaje de 0 a 10 por criterio, y cada caso de uso fija un presupuesto mensual y un puntaje mínimo (letra, pág. 1, Contexto).

1.2. Las ocho funcionalidades

#	Funcionalidad (letra, pág. 1)	Dónde vive en el código
1	Registrar criterios con nombre único y peso positivo	`main.registrar_criterio`, `Sistema.registrar_criterio`
2	Registrar opciones de tipo API o plan mensual, con puntaje por criterio	`main.registrar_opcion`, `Sistema.registrar_servicio_api` y `registrar_plan_mensual`
3	Registrar casos de uso con presupuesto y puntaje mínimo	`main.registrar_caso`, `Sistema.registrar_caso`
4	Evaluación individual de una opción para un caso	`main.registrar_evaluacion`, `Sistema.evaluar`
5	Comparar todas las opciones para un caso y asignar estados	`main.comparar`, `Sistema.comparar_caso`
6	Actualizar el presupuesto de un caso y recalcular	`main.actualizar_presupuesto`, `Sistema.actualizar_presupuesto`
7	Actualizar precio o consumo de una opción y recalcular	`main.actualizar_precio`, `Sistema.actualizar_costos_opcion`
8	Listar criterios, opciones, casos, evaluaciones y ranking	`main.listar_*`, `Sistema.evaluaciones_de_caso`, `Sistema.ranking_caso`

1.3. Las clases obligatorias

La letra exige siete clases (pág. 2, Clases): `Sistema`, `OpcionIA` (abstracta), `ServicioAPI` y `PlanMensual` (derivadas), `Criterio`, `CasoUso` y `Evaluacion`. Además pide al menos una excepción propia, que no cuenta como clase del dominio (misma sección). La estructura de carpetas está fijada (pág. 5, Estructura de la solución): `main.py`, `README.md`, `entities/` con un archivo por clase (`OpcionIA.py` incluye las dos derivadas) y `exceptions/ErrorLaboratorio.py`. El diagrama de clases de la página 2 se transcribe y se compara con el código en la sección 2.5.

1.4. El menú

El menú principal tiene cinco opciones: Registrar, Comparar, Actualizar, Listar y Salir (pág. 3, Menú principal). En cada submenú se permanece hasta elegir Salir; una entrada incorrecta muestra un mensaje y permite intentar de nuevo (misma página).

1.5. Las reglas que más se preguntan

Regla	Letra
Nombre de criterio único sin distinguir mayúsculas; peso entero positivo; los pesos no necesitan sumar 100	pág. 3, Criterio
Todos los criterios antes de la primera opción; después no se aceptan nuevos; hace falta al menos uno para registrar opciones	pág. 3, Criterio
Costo de Servicio API = (consultas / 1000) × precio por cada 1000. Ejemplo: (5000 / 1000) × 40 = USD 200	pág. 3, Opción de IA
Costo de Plan mensual = usuarios × costo por usuario	pág. 3, Opción de IA
Un puntaje entero de 0 a 10 por criterio, sin omisiones ni duplicados; ambos tipos implementan `costo_estimado()`	pág. 3, Opción de IA
Caso: presupuesto ≥ 0, mínimo entre 0 y 10; registrar un caso no crea evaluaciones	pág. 3, Caso de uso
Evaluación duplicada (misma opción y caso): se informa y no se crea	pág. 4, Evaluación individual
Puntaje total = suma(puntaje × peso) / suma de pesos, con dos decimales	pág. 4, Evaluación individual
Estados en orden: «Fuera de presupuesto», «Puntaje insuficiente», «Recomendada»	pág. 4, Evaluación individual
Costo igual al presupuesto y puntaje igual al mínimo son válidos	pág. 4, Evaluación individual
Comparar reemplaza todas las evaluaciones del caso, incluso las individuales; conserva las de otros casos; los códigos no se reutilizan	pág. 4, Comparar opciones
Actualizar presupuesto: recalcula estados sin crear evaluaciones ni cambiar fechas	pág. 4, Presupuesto de un caso
Actualizar precio: recalcula costo y estado en todos los casos, sin cambiar códigos ni fechas; los puntajes no cambian	pág. 4, Precio o consumo
Ranking: solo «recomendada», mayor puntaje primero; empate por menor costo y después por menor código de opción	pág. 5, Listar datos (ranking)
Validar tipos, vacíos, rangos, códigos inexistentes y duplicados, y pedir de nuevo sin terminar	pág. 5, Consideraciones generales
Fecha y hora con `datetime`; sin persistencia ni internet; Python ≥ 3.10	pág. 5, Consideraciones generales
Puntajes como `{codigo_criterio: puntaje}`; las evaluaciones guardan referencias a la opción y al caso	pág. 2, debajo del diagrama

2. Mapa del programa

2.1. Archivos y responsabilidades

Archivo	Contiene	Responsabilidad
`main.py`	funciones de lectura, menús, listados	Toda la conversación con el usuario: pide datos, muestra resultados y mensajes
`entities/Sistema.py`	clase `Sistema`	Dueño de las cuatro listas y de los contadores de códigos; coordina las operaciones
`entities/OpcionIA.py`	`OpcionIA` (abstracta), `ServicioAPI`, `PlanMensual`	Datos de una opción y su fórmula de costo
`entities/Criterio.py`	clase `Criterio`	Un aspecto a evaluar y su peso
`entities/CasoUso.py`	clase `CasoUso`	Una necesidad con su presupuesto y su puntaje mínimo
`entities/Evaluacion.py`	clase `Evaluacion`	Une una opción con un caso; calcula puntaje y costo, y decide el estado
`exceptions/ErrorLaboratorio.py`	clase `ErrorLaboratorio`	La excepción propia para reglas violadas

Archivo	Contiene	Responsabilidad
`README.md`	texto plano	Número de equipo (5), requisitos, cómo ejecutar, estructura

2.2. Quién llama a quién

```
          usuario (teclado y pantalla)
                      |
                    main.py        leer_*, seleccionar_*, registrar_*, listar_*,
                      |            comparar, actualizar_*, mostrar_*
                      |  llama métodos y lee propiedades
                      v
                    Sistema        listas privadas + contadores de códigos
          /           |      \           \
    Criterio    CasoUso    OpcionIA     Evaluacion ---> referencia a una OpcionIA
                             ^    ^                 ---> referencia a un CasoUso
                ServicioAPI    PlanMensual

  ErrorLaboratorio: la lanza Sistema (raise); la captura main.py (except)
```

`main.py` nunca crea un `Criterio` o una `Evaluacion` por su cuenta: siempre se lo pide a `Sistema`. `Sistema` nunca muestra nada: devuelve objetos o lanza `ErrorLaboratorio`.

2.3. Por qué `main.py` habla y `entities/` no

Ningún archivo de `entities/` ni de `exceptions/` usa `input()` o `print()` (se revisó todo el código). Hay tres motivos:

1. **Coincide con el diagrama** (sección 2.5): `Sistema` no tiene responsabilidades de consola.
2. **No se repite la lectura**: `leer_entero` se escribe una vez y la usan todos los menús y registros.
3. **Separa qué hace el sistema de cómo se conversa**: cada mitad se entiende y se corrige por separado.

Ejemplo concreto: `Sistema.validar_nombre` no imprime «nombre repetido»; lanza `ErrorLaboratorio` con el mensaje, y `main.leer_nombre_nuevo` lo muestra y vuelve a pedir el nombre.

Las diapositivas 10, 11 y 13 del profesor muestran `input()` dentro de `Sistema`. La diapositiva 9 muestra `registrar_criterio(self, nombre, peso)` recibiendo los datos ya leídos. El equipo aplicó esa misma forma a todas las operaciones para que toda la lectura de datos quede en `main.py` y `Sistema` tenga solo los métodos del dominio, como en el diagrama.

2.4. Una operación de punta a punta: comparar

1. `main.comparar` comprueba que haya opciones; si no hay, muestra «No hay opciones registradas para comparar.» y vuelve.
2. `seleccionar_caso` lista los casos y pide un código; `Sistema.buscar` devuelve el caso o lanza `ErrorLaboratorio`, y entonces se vuelve a pedir.
3. `main` llama a `sistema.comparar_caso(caso)`.
4. `Sistema` arma una lista con las evaluaciones de los otros casos y descarta las de este.
5. Por cada opción llama a `__crear_evaluacion`, que crea la `Evaluacion` con el próximo código y `datetime.now()`.
6. Cada evaluación ejecuta `recalcular`: pide el costo a la opción, el puntaje a sí misma y fija el estado.

7. `main` muestra las evaluaciones del caso con `mostrar_evaluaciones`.

Resumen para decir en voz alta: *main.py conversa, Sistema coordina, Evaluacion decide el estado y cada opción calcula su costo.*

2.5. El diagrama de la letra frente al código

El diagrama de clases está en la página 2 de la letra; es conveniente mirarlo una vez antes de la defensa. Esta es su transcripción (− privado, + público; las flechas indican la multiplicidad de cada relación):

```
Sistema                          --> OpcionIA 0..*, Criterio 0..*,
  - opciones: [OpcionIA]             CasoUso 0..*, Evaluacion 0..*
  - criterios: [Criterio]
  - casos: [CasoUso]
  - evaluaciones: [Evaluacion]
  + comparar_caso(): void
  + recalcular(): void

«abstracta» OpcionIA             ServicioAPI (hereda)           PlanMensual (hereda)
  - codigo: int                    - costo_por_1000: float        - usuarios: int
  - nombre: str                    - consultas_estimadas: int     - costo_por_usuario: float
  - puntajes: dict                 + costo_estimado(): float      + costo_estimado(): float
  + costo_estimado(): float

Evaluacion                       --> OpcionIA 1, CasoUso 1
  - codigo: int                  Criterio                 CasoUso
  - opcion: OpcionIA               - codigo: int            - codigo: int
  - caso: CasoUso                  - nombre: str            - nombre: str
  - fecha: datetime                - peso: int              - presupuesto: float
  - estado: str                                             - puntaje_minimo: float
  + puntaje_total(): float
  + costo_total(): float
```

Todos los atributos del diagrama están en el código con el mismo nombre. Lo que el código agrega o cambia, y por qué:

En el diagrama	En el código	Por qué
`Sistema`: cuatro listas, `comparar_caso()` y `recalcular()`	cuatro contadores, tres propiedades de lectura y trece métodos públicos: los dos del diagrama y además `validar_nombre`, `buscar`, cuatro `registrar_*`, `evaluar`, dos `actualizar_*`, `evaluaciones_de_caso` y `ranking_caso`	La letra pide que `Sistema` asigne códigos y coordine las operaciones; el diagrama muestra solo los métodos centrales
`comparar_caso()` y `recalcular()` sin parámetros	`comparar_caso(caso)`, `recalcular(evaluaciones)`	Necesitan saber qué caso comparar y qué evaluaciones recalcular
`puntaje_total()` sin parámetros	`puntaje_total(criterios)`	`Evaluacion` no tiene relación con `Criterio` en el diagrama y necesita los pesos
`Evaluacion` sin `recalcular`	`recalcular(criterios)` en `Evaluacion`	Es la clase que tiene la opción, el caso y el estado; `Sistema.recalcular` le delega

En el diagrama	En el código	Por qué
`ServicioAPI` y `PlanMensual` solo con `costo_estimado()`	además `tipo`, `descripcion_costos()`, `actualizar_costos()`	Los piden el listado de opciones (letra, pág. 4) y la actualización de precio
Sin setters	un único setter: `CasoUso.presupuesto`	Es el único dato suelto que la letra pide actualizar
Atributos con – (privados)	`__atributo` con `@property`	Es la forma que usa el curso (diapositiva 8): el doble guion bajo hace que Python renombre el atributo (sección 3.4).

La excepción `ErrorLaboratorio` no figura en el diagrama; es coherente con la letra, que aclara que las excepciones no cuentan como clases del dominio.

2.6. Datos de ejemplo

Todos los ejemplos con números de esta guía usan los datos de la Parte A de la demostración (capítulo 8).

Criterios: Precisión (código 1, peso 3), Velocidad (código 2, peso 2) y Soporte (código 3, peso 1).

Opción	Tipo	Datos de costo	Puntajes (Precisión, Velocidad, Soporte)
1. Claude	Servicio API	USD 40 por cada 1000 consultas, 5000 consultas	8, 7, 9
2. OpenAI	Plan mensual	10 usuarios, USD 15 por usuario	6, 7, 5
3. Gemini	Servicio API	USD 30 por cada 1000 consultas, 10000 consultas	9, 8, 8

Casos: 1 «Atención al cliente», presupuesto USD 250 y mínimo 7; 2 «Análisis de documentos», presupuesto USD 300 y mínimo 7.5.

3. Conceptos de POO con el código real

3.1. Clase abstracta: `OpcionIA`

Una clase abstracta describe algo general que no puede existir por sí solo. Nadie contrata «una opción de IA» sin más: contrata un servicio por consultas o un plan por usuarios. Sin el tipo no hay fórmula de costo.

`entities/OpcionIA.py`, líneas 1 a 10:

```python
from abc import ABC, abstractmethod


class OpcionIA(ABC):
    """Opción de IA abstracta. Los puntajes son un diccionario {codigo_criterio: puntaje}."""

    def __init__(self, codigo, nombre, puntajes):
        self.__codigo = codigo
        self.__nombre = nombre
        self.__puntajes = puntajes
```

`entities/OpcionIA.py`, líneas 24 a 27:

```
    @abstractmethod
    def costo_estimado(self):
        """Devuelve el costo mensual estimado; cada clase derivada escribe el suyo."""
        pass
```

- `ABC` (de la biblioteca estándar `abc`) activa el control de métodos abstractos. Por sí sola no impide crear objetos: lo impide que, además, haya al menos un método marcado con `@abstractmethod` sin implementar.
- `@abstractmethod` obliga a cada derivada a escribir `costo_estimado()`. El `pass` no hace nada: indica que la clase base no tiene fórmula propia (la docstring sola también bastaría como cuerpo). En este programa ninguna derivada llama a la versión del padre.
- Python impide crear el objeto. Ejecutado: `OpcionIA(1, "x", {})` produce `TypeError: Can't instantiate abstract class OpcionIA without an implementation for abstract method 'costo_estimado'`.

3.2. Herencia: `ServicioAPI` y `PlanMensual`

`entities/OpcionIA.py`, líneas 30 a 36:

```
class ServicioAPI(OpcionIA):
    """Opción que se paga por consultas a una API."""

    def __init__(self, codigo, nombre, puntajes, costo_por_1000, consultas_estimadas):
        super().__init__(codigo, nombre, puntajes)
        self.__costo_por_1000 = costo_por_1000
        self.__consultas_estimadas = consultas_estimadas
```

`class ServicioAPI(OpcionIA)` significa «un ServicioAPI *es una* OpcionIA». `super().__init__(...)` delega en el constructor de la clase padre la carga de `codigo`, `nombre` y `puntajes`; la derivada solo agrega sus dos datos propios. `PlanMensual` (líneas 63 a 69) hace lo mismo con `usuarios` y `costo_por_usuario`. Ambas heredan las propiedades `codigo`, `nombre` y `puntajes` sin escribirlas de nuevo.

3.3. Polimorfismo: `costo_estimado()`

El mismo mensaje produce el cálculo adecuado según el objeto que lo recibe.

`entities/OpcionIA.py`, líneas 50 a 52 (`ServicioAPI`):

```
    def costo_estimado(self):
        """Costo mensual = (consultas mensuales / 1000) × precio por cada 1000."""
        return self.__consultas_estimadas / 1000 * self.__costo_por_1000
```

`entities/OpcionIA.py`, líneas 83 a 85 (`PlanMensual`):

```
    def costo_estimado(self):
        """Costo mensual = usuarios × costo por usuario."""
        return self.__usuarios * self.__costo_por_usuario
```

Quien llama no pregunta el tipo. `entities/Evaluacion.py`, líneas 48 y 49:

```
    def costo_total(self):
        return self.__opcion.costo_estimado()
```

Dónde está el polimorfismo: en esa línea. `Evaluacion` no tiene un `if` por tipo de opción; un tercer tipo no la obligaría a cambiar. Lo mismo ocurre con `tipo`, `descripcion_costos()` (usados por

`main.listar_opciones`) y `actualizar_costos()` (usado por `Sistema.actualizar_costos_opcion`): cada derivada tiene su versión y el llamador no distingue. Esos tres miembros no están declarados en `OpcionIA`: funcionan porque las dos derivadas los escriben con el mismo nombre (sección 4.2).

El único `isinstance` está en `main.actualizar_precio` (línea 278). No contradice el polimorfismo: la consola necesita saber *qué preguntar* (precio y consultas, o usuarios y costo por usuario). Una vez leídos los dos valores, la actualización en el dominio es polimórfica: `opcion.actualizar_costos(primer_valor, segundo_valor)`.

3.4. Encapsulamiento: atributos privados, `@property` y el único setter

Todos los atributos de instancia llevan doble guion bajo (`self.__peso`). En `Criterio`, `CasoUso`, `OpcionIA` y sus derivadas, y `Evaluacion`, cada uno se lee con una propiedad; `Sistema` solo expone `criterios`, `opciones` y `casos`: las evaluaciones se consultan con `evaluaciones_de_caso` y los contadores no se exponen. Las tres constantes de estado de `Evaluacion` son atributos de clase públicos.

`entities/Criterio.py`, líneas 1 a 19:

```
class Criterio:
    """Criterio de evaluación con un peso entero."""

    def __init__(self, codigo, nombre, peso):
        self.__codigo = codigo
        self.__nombre = nombre
        self.__peso = peso

    @property
    def codigo(self):
        return self.__codigo

    @property
    def nombre(self):
        return self.__nombre

    @property
    def peso(self):
        return self.__peso
```

- Una **propiedad** es un método que se lee como un atributo, sin paréntesis: `criterio.peso`.
- Sin setter, el dato es de solo lectura. Ejecutado: `criterio.peso = 5` produce `AttributeError: property 'peso' of 'Criterio' object has no setter`.
- El doble guion bajo activa el *name mangling*: Python renombra `__peso` a `_Criterio__peso`. Por eso `criterio.__peso` desde afuera da `AttributeError`. No es un candado absoluto (`criterio._Criterio__peso` devuelve 3), sino una protección contra modificaciones accidentales.
- Por eso una derivada tampoco ve los atributos privados del padre: dentro de `ServicioAPI`, `self.__codigo` se traduce a `self._ServicioAPI__codigo`, que no existe (el atributo se llama `_OpcionIA__codigo`). Si una derivada necesita el código, usa la propiedad heredada `self.codigo`.

El único setter del programa está en `CasoUso`, porque la letra pide actualizar el presupuesto (pág. 4, Presupuesto de un caso). `entities/CasoUso.py`, líneas 18 a 24:

```
    @property
    def presupuesto(self):
        return self.__presupuesto

    @presupuesto.setter
    def presupuesto(self, nuevo_presupuesto):
        self.__presupuesto = nuevo_presupuesto
```

Los costos de una opción también cambian, pero son dos valores que cambian juntos, así que se modifican con un método: `actualizar_costos(...)` (líneas 58 a 60 y 91 a 93 de `OpcionIA.py`). Regla para recordar: *solo se puede modificar lo que la letra pide actualizar.*

3.5. Excepción propia: `ErrorLaboratorio`

`exceptions/ErrorLaboratorio.py`, líneas 1 a 3:

```
class ErrorLaboratorio(Exception):
    """Error de una regla del laboratorio (nombre repetido, código inexistente, evaluación repetida)."""
    pass
```

Hereda de `Exception`, así que se puede lanzar con `raise` y capturar con `except`. Se lanza en tres métodos de `Sistema`: `validar_nombre`, `buscar` y `evaluar`. Se captura en cuatro funciones de `main.py`, que es donde se puede resolver: `leer_nombre_nuevo`, `seleccionar_opcion`, `seleccionar_caso` y `registrar_evaluacion`. En los cuatro se muestra `Error: <mensaje>`.

Por qué no se captura `Exception`: `except Exception` atraparía también los errores de programación (un `TypeError`, un `KeyError`) y los mostraría como si fueran errores del usuario, escondiendo el problema. Se capturan solo los errores esperados: `ValueError` al convertir texto a número y `ErrorLaboratorio` al violar una regla. Se revisó todo el código: no hay `except Exception` ni `except:` en el proyecto.

Por qué una excepción propia y no `ValueError`: `ValueError` es lo que lanza `int("abc")`. Si las reglas también usaran `ValueError`, no se podría distinguir «escribió letras» de «el nombre ya existe».

3.6. Diccionario de puntajes

La letra fija el formato `{codigo_criterio: puntaje}` (pág. 2, debajo del diagrama). `main.py`, líneas 61 a 67:

```
def pedir_puntajes(criterios):
    """Pide un puntaje entero de 0 a 10 por cada criterio y devuelve {codigo: puntaje}."""
    puntajes = {}
    for criterio in criterios:
        puntajes[criterio.codigo] = leer_entero(
            f"Puntaje para {criterio.nombre} (0 a 10): ", 0, 10)
    return puntajes
```

Para Claude, con los datos de la sección 2.6, queda `{1: 8, 2: 7, 3: 9}`. La clave es el código porque es único y nunca cambia. «Sin omisiones ni duplicados» (letra, pág. 3, Opción de IA) se cumple por construcción: el bucle recorre todos los criterios una vez, y un diccionario no puede tener dos valores para la misma clave.

3.7. Referencias a objetos en `Evaluacion`

`entities/Evaluacion.py`, líneas 8 a 13:

```
    def __init__(self, codigo, opcion, caso, fecha):
        self.__codigo = codigo
        self.__opcion = opcion
        self.__caso = caso
        self.__fecha = fecha
        self.__estado = ""
```

`self.__opcion` no es una copia ni un código: apunta al mismo objeto que está en la lista de `Sistema` (letra, pág. 2, debajo del diagrama). Consecuencia: cuando cambia el precio de Claude, todas sus evaluaciones ven el precio nuevo sin que nadie las actualice una por una. Lo único que hay que rehacer es el estado, porque el estado sí se guarda. El estado empieza como cadena vacía y `Sistema` siempre llama a `recalcular` antes de guardar la evaluación, así que ninguna evaluación guardada queda sin estado.

3.8. `datetime`

`entities/Sistema.py` importa `from datetime import datetime` (línea 1) y fija la fecha al crear cada evaluación con `datetime.now()` (línea 80). `main.py` la muestra con `evaluacion.fecha.strftime(FORMATO_FECHA)`, donde `FORMATO_FECHA = "%d/%m/%Y %H:%M:%S"` (línea 6). Ejemplo real: `06/11/2026 18:04:46`.

3.9. `match/case`

`main.py`, líneas 299 a 310:

```
        match leer_entero("Seleccione una opción: ", 1, 5):
            case 1:
                menu_registrar(sistema)
            case 2:
                comparar(sistema)
            case 3:
                menu_actualizar(sistema)
            case 4:
                menu_listar(sistema)
            case 5:
                print("Programa finalizado.")
                break
```

`match` compara el valor con cada `case` y ejecuta el primero que coincide. Es equivalente a una cadena `if/elif`. Existe desde Python 3.10, que es el mínimo que fija la letra (pág. 5, Consideraciones generales). No hace falta un `case _` (el «cualquier otro») porque `leer_entero(..., 1, 5)` garantiza que el valor está entre 1 y 5. Por qué `match` y no el `if/elif` de las diapositivas: fue una elección del equipo; la letra exige Python 3.10 o mayor, donde `match` existe, y para un menú de valores fijos se lee como una lista vertical.

3.10. Contadores de códigos y por qué no `len(lista) + 1`

`entities/Sistema.py`, líneas 13 a 21:

```
def __init__(self):
    self.__criterios = []
    self.__opciones = []
    self.__casos = []
    self.__evaluaciones = []
    self.__proximo_codigo_criterio = 1
    self.__proximo_codigo_opcion = 1
    self.__proximo_codigo_caso = 1
    self.__proximo_codigo_evaluacion = 1
```

Cada entidad tiene su contador, que solo aumenta. Con `len(lista) + 1` se repetirían códigos, porque comparar *elimina* evaluaciones. Ejemplo real de la demostración (capítulo 8; la regla de comparar se detalla en la sección 6.5): la evaluación individual recibe el código 1; al comparar el caso 1 se elimina y se crean la 2, la 3 y la 4. La lista queda con 3 elementos, así que `len + 1` daría 4, un código que ya existe. El contador da 5, que es lo correcto. La letra lo prohíbe expresamente: «Los códigos no se reutilizan» (pág. 4, Comparar opciones para un caso).

4. Recorrido clase por clase

4.1. `Criterio` y `CasoUso`

Son clases de datos. `Criterio` guarda `codigo`, `nombre` y `peso`, todos de solo lectura (código completo en la sección 3.4). `CasoUso` guarda `codigo`, `nombre`, `presupuesto` y `puntaje_minimo`; solo `presupuesto` tiene setter. Ninguna valida nada: reciben datos ya validados por `main.py`.

4.2. `OpcionIA`, `ServicioAPI` y `PlanMensual`

Miembro	`ServicioAPI`	`PlanMensual`
Datos propios	`costo_por_1000`, `consultas_estimadas`	`usuarios`, `costo_por_usuario`
`tipo` (propiedad)	`"Servicio API"`	`"Plan mensual"`
`costo_estimado()`	consultas / 1000 × precio	usuarios × costo por usuario
`descripcion_costos()`	`USD 40.00 por cada 1000 consultas, 5000 consultas estimadas`	`10 usuarios, USD 15.00 por usuario`
`actualizar_costos(a, b)`	cambia precio y consultas	cambia usuarios y costo por usuario

Los textos de `descripcion_costos()` son los que muestra el listado de opciones de la demostración. Solo `costo_estimado()` es abstracto. Los otros tres (`tipo`, `descripcion_costos()` y `actualizar_costos()`) están en las dos derivadas pero no en `OpcionIA`: si un tercer tipo los olvidara, Python lo dejaría crear y el error aparecería recién al listar o actualizar (`AttributeError`). Declararlos abstractos sería la forma de obligar a escribirlos; no se hizo porque la letra (pág. 3, Opción de IA) y el diagrama solo exigen `costo_estimado()`.

4.3. `Evaluacion`

Guarda `codigo`, `opcion`, `caso`, `fecha` y `estado`. Tiene tres constantes de clase para los estados (líneas 4 a 6): `RECOMENDADA`, `FUERA_DE_PRESUPUESTO` y `PUNTAJE_INSUFICIENTE`. Usar constantes evita escribir el texto a mano en varios lugares y equivocarse en una letra.

`entities/Evaluacion.py`, líneas 35 a 46:

```python
def puntaje_total(self, criterios):
    """Devuelve suma(puntaje × peso) / suma de pesos, sin redondear."""
    puntajes = self.__opcion.puntajes
    suma_ponderada = 0
    suma_pesos = 0
    posicion = 0
    while posicion < len(criterios):
        criterio = criterios[posicion]
        suma_ponderada += puntajes[criterio.codigo] * criterio.peso
        suma_pesos += criterio.peso
        posicion += 1
    return suma_ponderada / suma_pesos
```

- Recibe la lista de criterios porque `Evaluacion` no conoce a `Criterio` y necesita los pesos (en el diagrama, `puntaje_total()` no tiene parámetros; ver 2.5).
- Busca cada puntaje por código en el diccionario de la opción.
- Devuelve el valor **sin redondear**; el redondeo a dos decimales es solo para mostrar.
- No puede dividir por cero: cada peso es al menos 1 y no se registran opciones sin criterios.
- El `while` con índice y el `for` recorren la lista completa y dan el mismo resultado; el programa usa `for` en todos los recorridos menos en este, que conserva la forma de la diapositiva 12 (`pedir_puntajes`, que en esa diapositiva usa `while`, se escribió con `for` como el resto). Si se pregunta por la diferencia: el `for` evita manejar el índice a mano, y el `while` muestra explícitamente cómo avanza la posición.

`entities/Evaluacion.py`, líneas 51 a 59:

```python
def recalcular(self, criterios):
    """Fija el estado en el orden de la letra; la igualdad con cada límite es válida."""
    # Redondeo para que 5 × 5.99 sea igual a 29.95.
    if round(self.costo_total() - self.__caso.presupuesto, 6) > 0:
        self.__estado = Evaluacion.FUERA_DE_PRESUPUESTO
    elif self.puntaje_total(criterios) < self.__caso.puntaje_minimo:
        self.__estado = Evaluacion.PUNTAJE_INSUFICIENTE
    else:
        self.__estado = Evaluacion.RECOMENDADA
```

Fuera del valor inicial `""` que pone el constructor, es el **único método que escribe el estado**. El orden `if/elif/else` reproduce el orden de la letra (pág. 4, Evaluación individual): primero el presupuesto, después el puntaje. El `> 0` y el < estrictos hacen que la igualdad sea válida (misma sección). El redondeo se explica en la sección 6.4.

4.4. `Sistema`

Tiene las cuatro listas y los cuatro contadores (sección 3.10). Expone tres propiedades de lectura (`criterios`, `opciones`, `casos`) y trece métodos públicos (el diagrama muestra solo dos; ver 2.5). Todos los métodos que trabajan sobre algo ya elegido reciben **objetos**, no códigos.

Validar y buscar. `entities/Sistema.py`, líneas 35 a 46:

```
def validar_nombre(self, registros, nombre):
    """Lanza ErrorLaboratorio si ya hay un registro con ese nombre, sin distinguir mayúsculas."""
    for registro in registros:
        if registro.nombre.lower() == nombre.lower():
            raise ErrorLaboratorio(f"Ya existe un registro llamado {nombre}.")

def buscar(self, registros, codigo):
    """Devuelve el registro con ese código o lanza ErrorLaboratorio si no existe."""
    for registro in registros:
        if registro.codigo == codigo:
            return registro
    raise ErrorLaboratorio(f"No existe un registro con código {codigo}.")
```

Los dos reciben la lista en la que trabajar, así que un solo método sirve para criterios, opciones y casos. `lower()` hace que «Precisión», «PRECISIÓN» y «precisión» sean el mismo nombre. La unicidad es por lista: un criterio y una opción pueden llamarse igual.

Registrar. Los cuatro métodos `registrar_*` siguen el patrón de las diapositivas 8 y 9: crear con el próximo código, agregar a la lista, avanzar el contador y devolver el objeto. `entities/Sistema.py`, líneas 48 a 53:

```
def registrar_criterio(self, nombre, peso):
    """Crea un criterio con el próximo código, lo guarda y lo devuelve."""
    criterio = Criterio(self.__proximo_codigo_criterio, nombre, peso)
    self.__criterios.append(criterio)
    self.__proximo_codigo_criterio += 1
    return criterio
```

Hay dos métodos para opciones, `registrar_servicio_api` y `registrar_plan_mensual`, porque cada tipo tiene datos distintos.

Crear, evaluar y comparar. `entities/Sistema.py`, líneas 78 a 105:

```
def __crear_evaluacion(self, opcion, caso):
    """Crea la evaluación con el próximo código y la fecha actual, la calcula y la guarda."""
    evaluacion = Evaluacion(self.__proximo_codigo_evaluacion, opcion, caso, datetime.now())
    evaluacion.recalcular(self.__criterios)
    self.__evaluaciones.append(evaluacion)
    self.__proximo_codigo_evaluacion += 1
    return evaluacion

def evaluar(self, opcion, caso):
    """Crea la evaluación de una opción para un caso; lanza ErrorLaboratorio si ya existe."""
    for evaluacion in self.evaluaciones_de_caso(caso):
        if evaluacion.opcion.codigo == opcion.codigo:
            raise ErrorLaboratorio(
                f"Ya existe una evaluación de {opcion.nombre} para {caso.nombre}.")
    return self.__crear_evaluacion(opcion, caso)

def comparar_caso(self, caso):
    """Reemplaza todas las evaluaciones del caso por una por cada opción registrada.

    Conserva las evaluaciones de los otros casos; el contador de códigos no se reinicia.
    """
    conservadas = []
    for evaluacion in self.__evaluaciones:
        if evaluacion.caso.codigo != caso.codigo:
            conservadas.append(evaluacion)
    self.__evaluaciones = conservadas
    for opcion in self.__opciones:
        self.__crear_evaluacion(opcion, caso)
```

- `__crear_evaluacion` es privado (doble guion bajo) y lo usan `evaluar` y `comparar_caso`. Así se cumple que la evaluación individual «utiliza las mismas reglas de cálculo que la comparación» (letra, pág. 4, Evaluación individual): las reglas están escritas una sola vez. Es privado para que nadie fuera de `Sistema` cree una evaluación salteando el control de duplicados de `evaluar` o el reemplazo de `comparar_caso`. Los demás métodos son públicos porque `main.py` los necesita.
- `evaluar` revisa el duplicado **antes** de crear. Si hay duplicado, el contador no avanza: un intento rechazado no consume código.
- `comparar_caso` arma una lista nueva con lo que se conserva en lugar de borrar mientras recorre. Borrar elementos de una lista mientras se la recorre con `for` hace que se salten elementos.

Actualizar y recalcular. `entities/Sistema.py`, líneas 107 a 124:

```
    def recalcular(self, evaluaciones):
        """Recalcula el estado de las evaluaciones indicadas; no cambia códigos, fechas ni puntajes."""
        for evaluacion in evaluaciones:
            evaluacion.recalcular(self.__criterios)

    def actualizar_presupuesto(self, caso, nuevo):
        """Cambia el presupuesto del caso y recalcula solo las evaluaciones de ese caso."""
        caso.presupuesto = nuevo
        self.recalcular(self.evaluaciones_de_caso(caso))

    def actualizar_costos_opcion(self, opcion, primer_valor, segundo_valor):
        """Cambia los costos de la opción y recalcula sus evaluaciones en todos los casos."""
        opcion.actualizar_costos(primer_valor, segundo_valor)
        afectadas = []
        for evaluacion in self.__evaluaciones:
            if evaluacion.opcion.codigo == opcion.codigo:
                afectadas.append(evaluacion)
        self.recalcular(afectadas)
```

`caso.presupuesto = nuevo` usa el setter de `CasoUso`. Ninguno de los dos crea evaluaciones ni toca fechas o códigos: solo vuelve a calcular el estado de las existentes. El presupuesto afecta a un caso; el precio afecta a una opción **en todos los casos**.

Consultar y ordenar. `entities/Sistema.py`, líneas 126 a 147:

```
    def evaluaciones_de_caso(self, caso):
        """Devuelve una lista nueva con las evaluaciones del caso indicado."""
        resultado = []
        for evaluacion in self.__evaluaciones:
            if evaluacion.caso.codigo == caso.codigo:
                resultado.append(evaluacion)
        return resultado

    def ranking_caso(self, caso):
        """Devuelve las evaluaciones recomendadas del caso, de mejor a peor.

        Orden: mayor puntaje sin redondear, menor costo (redondeado a 6 decimales
        para ignorar el ruido de los decimales) y menor código de opción.
        """
        recomendadas = []
        for evaluacion in self.evaluaciones_de_caso(caso):
            if evaluacion.estado == Evaluacion.RECOMENDADA:
                recomendadas.append(evaluacion)
        return sorted(recomendadas, key=lambda evaluacion: (
            -evaluacion.puntaje_total(self.__criterios),
            round(evaluacion.costo_total(), 6),
            evaluacion.opcion.codigo))
```

`lambda evaluacion: (...)` es una función sin nombre escrita en una línea. Recibe una evaluación y devuelve la tupla por la que se ordena. Equivale a escribir, dentro del método, `def clave(evaluacion): return (-evaluacion.puntaje_total(self.__criterios), round(evaluacion.costo_total(), 6), evaluacion.opcion.codigo)` y pasar `key=clave`. `sorted` llama a esa función una vez por cada evaluación y ordena según lo que devuelve.

`sorted` ordena de menor a mayor según la clave. La clave es una tupla: Python compara el primer elemento, y si empatan, el segundo, y así. El signo menos en el puntaje invierte ese criterio para que el mayor puntaje quede primero; costo y código quedan de menor a mayor, como pide la letra (pág. 5, Listar datos). `sorted` devuelve una lista nueva y no modifica nada. `main.listar_casos` usa `len(sistema.ranking_caso(caso))` para contar las recomendadas de cada caso.

5. `main.py`: menús y lectura de datos

5.1. Los menús

`main()` crea **un único** `Sistema` (línea 290) y se lo pasa a cada función. Cada menú es un `while True` que muestra las opciones, lee con `leer_entero(..., 1, N)` y despacha con `match`. La opción Salir hace `break`: en un submenú vuelve al menú principal; en el principal imprime «Programa finalizado.» y termina. Después de cada acción el bucle vuelve a mostrar el mismo submenú, como pide la letra (pág. 3). Los textos y números de los menús son los de la letra (pág. 3, Menú principal).

La última línea, `if __name__ == "__main__": main()` (`main.py`, líneas 379 y 380), ejecuta el menú solo cuando se corre `python3 main.py`; si otro archivo importara `main.py`, el menú no arrancaría solo. Los menús usan `while True` y salen con `break` en lugar de `while opcion != 5` (diapositiva 7): así la variable `opcion` no necesita un valor inicial ficticio y la salida queda escrita en el mismo `case` que la elige.

5.2. Las funciones de lectura

`main.py`, líneas 9 a 22:

```
def leer_entero(mensaje, minimo=None, maximo=None):
    """Pide un entero hasta que sea válido y esté dentro del rango indicado."""
    while True:
        try:
            valor = int(input(mensaje))
        except ValueError:
            print("Ingrese un número entero.")
            continue
        if minimo is not None and valor < minimo:
            print(f"Ingrese un número mayor o igual que {minimo}.")
        elif maximo is not None and valor > maximo:
            print(f"Ingrese un número menor o igual que {maximo}.")
        else:
            return valor
```

- `int(...)` lanza `ValueError` si el texto no es un entero (`abc`, vacío, `2.5`). Se captura, se avisa y `continue` vuelve a preguntar.
- `minimo` y `maximo` son opcionales (`None` = sin límite). Así la misma función sirve para menús (1 a 5), pesos (≥ 1), puntajes (0 a 10) y códigos (sin rango; el código lo valida `Sistema.buscar`).
- `leer_flotante` (líneas 25 a 38) tiene la misma forma con `float(...)` y el mensaje «Ingrese un número.».

`main.py`, líneas 41 a 58:

```python
def leer_texto(mensaje):
    """Pide un texto hasta que no esté vacío después de quitar los espacios."""
    while True:
        texto = input(mensaje).strip()
        if texto:
            return texto
        print("El texto no puede estar vacío.")


def leer_nombre_nuevo(sistema, mensaje, registros):
    """Pide un nombre y lo repite mientras ya exista en los registros."""
    while True:
        nombre = leer_texto(mensaje)
        try:
            sistema.validar_nombre(registros, nombre)
            return nombre
        except ErrorLaboratorio as error:
            print(f"Error: {error}")
```

`strip()` quita los espacios de los extremos: `"  Hola  "` se guarda como `"Hola"` y un texto de solo espacios queda vacío y se rechaza. `leer_nombre_nuevo` combina las dos capas: la consola rechaza el vacío y `Sistema` rechaza el duplicado. El nombre se valida apenas se escribe, antes de pedir el peso o los costos, para que el usuario no cargue datos en vano.

Por qué `leer_*` capturan solo `ValueError`: es el único error esperable al convertir lo que se tecleó. Cualquier otro error sería un defecto del programa y no debe esconderse.

5.3. Precondiciones antes de pedir datos

Algunas acciones no tienen sentido en ciertos estados. `main.py` las comprueba **antes** de pedir cualquier dato y vuelve al submenú con un mensaje:

Acción	Condición	Mensaje
Registrar criterio	no debe haber opciones	«Ya hay opciones registradas: no se admiten nuevos criterios.»
Registrar opción	debe haber al menos un criterio	«Debe existir al menos un criterio para registrar opciones.»
Evaluación individual	debe haber casos (y opciones)	«No hay casos de uso registrados para seleccionar.»
Comparar	debe haber opciones	«No hay opciones registradas para comparar.»
Cualquier selección	debe haber registros	«No hay opciones registradas para seleccionar.» o el de casos

5.4. Reparto de la validación entre consola y dominio

Cada regla se valida en **una sola capa**. Una regla escrita dos veces puede quedar distinta en cada lugar.

Dato	Regla	Dónde se valida
Opción de menú y tipo de opción	entero en el rango mostrado	`leer_entero(..., 1, N)`
Nombre	no vacío	`leer_texto`

Dato	Regla	Dónde se valida
Nombre	único en su lista, sin distinguir mayúsculas	`Sistema.validar_nombre`
Peso, consultas, usuarios	entero ≥ 1	`leer_entero(..., 1)`
Puntajes	entero 0 a 10, uno por criterio	`pedir_puntajes`
Precio, costo por usuario, presupuesto	número ≥ 0	`leer_flotante(..., 0)`
Puntaje mínimo	número 0 a 10	`leer_flotante(..., 0, 10)`
Código	debe existir	`Sistema.buscar`
Evaluación	par opción y caso no repetido	`Sistema.evaluar`

Criterio práctico: **la consola valida forma y rango** (lo que se puede decidir mirando solo el dato); `Sistema` **valida lo que exige mirar lo ya registrado** (duplicados, existencia). Las entidades reciben datos ya validados y no repiten controles.

5.5. Qué pasa con una entrada inválida

Todas estas respuestas se observaron ejecutando el programa. En ningún caso termina.

Se escribe	Dónde	Respuesta
`abc`	cualquier menú	«Ingrese un número entero.»
`9`	menú principal	«Ingrese un número menor o igual que 5.»
`2.5`	donde se espera entero	«Ingrese un número entero.»
`1,5` (coma decimal)	donde se espera decimal	«Ingrese un número.» (hay que usar punto: `1.5`)
vacío o solo espacios	nombre	«El texto no puede estar vacío.»
`precisión` ya existiendo «Precisión»	nombre de criterio	«Error: Ya existe un registro llamado precisión.» y vuelve a pedir el nombre
`0`	peso, consultas, usuarios	«Ingrese un número mayor o igual que 1.»
`11`	puntaje	«Ingrese un número menor o igual que 10.»
`99`	código de opción o caso	«Error: No existe un registro con código 99.» y vuelve a pedir el código
misma opción y caso	evaluación individual	«Error: Ya existe una evaluación de Claude para Atención al cliente.» y vuelve al submenú

5.6. Listados

Listado	Campos que pide la letra (págs. 4 y 5)	Función de `main.py`	Si no hay datos
Criterios	código, nombre, peso	`listar_criterios`	«No hay criterios registrados.»
Opciones	código, nombre, tipo, datos de costo, costo mensual, puntajes	`listar_opciones` y `texto_puntajes`	«No hay opciones registradas.»
Casos	código, nombre, presupuesto, mínimo, recomendadas	`listar_casos` (cuenta con `len(ranking_caso(caso))`)	«No hay casos de uso registrados.»
Evaluaciones de un caso	código, fecha, opción, presupuesto actual, costo, puntaje, estado	`listar_evaluaciones` y `mostrar_evaluaciones`	«No hay evaluaciones para» y el nombre del caso

Listado	Campos que pide la letra (págs. 4 y 5)	Función de `main.py`	Si no hay datos
Ranking	posición, opción, puntaje, costo	`listar_ranking`	«No hay evaluaciones recomendadas para» y el nombre del caso

`mostrar_evaluacion` (una) responde a la evaluación individual e incluye el caso; `mostrar_evaluaciones` (varias) responde al listado y a la comparación, e incluye el presupuesto actual del caso, como pide la letra. `seleccionar_opcion` y `seleccionar_caso` devuelven `None` cuando no hay registros, y quien las llama vuelve al submenú con `if opcion is None: return` (o `caso`). La letra pide «mostrar todos los criterios» al registrar una opción: `pedir_puntajes` muestra el nombre de cada criterio al pedir su puntaje.

6. Reglas de negocio con números verificados

Los datos son los de la sección 2.6 (Parte A de la demostración).

6.1. Costo según el tipo

Opción	Tipo	Cálculo	Costo mostrado
Claude	Servicio API	5000 / 1000 × 40	USD 200.00
OpenAI	Plan mensual	10 × 15	USD 150.00
Gemini	Servicio API	10000 / 1000 × 30	USD 300.00

El de Claude es el ejemplo de la letra (pág. 3, Opción de IA).

6.2. Puntaje ponderado

Criterio	Peso	Claude	OpenAI	Gemini
Precisión	3	8 × 3 = 24	6 × 3 = 18	9 × 3 = 27
Velocidad	2	7 × 2 = 14	7 × 2 = 14	8 × 2 = 16
Soporte	1	9 × 1 = 9	5 × 1 = 5	8 × 1 = 8
Suma	6	47	37	51
Puntaje total		47 / 6 = 7.8333…	37 / 6 = 6.1666…	51 / 6 = 8.5
Se muestra		**7.83**	**6.17**	**8.50**

6.3. Orden de los estados

Comparar el caso 1 (presupuesto 250, mínimo 7) da los tres estados:

Opción	¿Costo > presupuesto?	¿Puntaje < mínimo?	Estado
Claude	200 > 250: no	7.83 < 7: no	Recomendada
OpenAI	150 > 250: no	6.17 < 7: sí	Puntaje insuficiente
Gemini	300 > 250: sí	(no se mira)	Fuera de presupuesto

Gemini tiene el mejor puntaje y aun así queda fuera: **primero se mira el presupuesto**. En el caso 2 (300 y 7.5), Claude es Recomendada (7.83 ≥ 7.5), OpenAI es Puntaje insuficiente y Gemini es Recomendada porque su costo, 300, es **igual** al presupuesto.

6.4. Igualdad en los límites y el `round(..., 6)`

La letra dice que un costo igual al presupuesto y un puntaje igual al mínimo son válidos (pág. 4, Evaluación individual).

Puntaje: se usa < estricto, así que 7 con mínimo 7 es Recomendada.

Costo: los números decimales (`float`) se guardan en binario y algunos no tienen representación exacta. Ejecutado en Python:

```
5 * 5.99                        -> 29.950000000000003
5 * 5.99 > 29.95                -> True
5 * 5.99 - 29.95                -> 3.552713678800501e-15
round(5 * 5.99 - 29.95, 6)      -> 0.0
```

Con `costo > presupuesto`, un plan de 5 usuarios a USD 5.99 quedaría «Fuera de presupuesto» con presupuesto USD 29.95, aunque la pantalla muestre USD 29.95 en ambos lados: violaría la letra. Con `round(costo - presupuesto, 6) > 0`, la diferencia de ruido (0.0000000000000035) se vuelve 0 y la opción es Recomendada. Una diferencia real de un centavo (0.01) sobrevive al redondeo a seis decimales y sigue quedando fuera. Se redondea la **diferencia** y solo para comparar; lo que se muestra se formatea aparte con dos decimales.

¿Por qué seis decimales y no dos? Con dos decimales se aceptaría un costo que supera el presupuesto por menos de medio centavo: 29.954 frente a 29.95 da una diferencia de 0.004, que redondeada a dos decimales es 0.00. Para montos como los del laboratorio, el ruido del `float` aparece alrededor del decimal quince; seis decimales lo eliminan y siguen detectando diferencias reales mucho menores que un centavo. Ejecutado: `round(29.954 - 29.95, 2)` da `0.0` y `round(29.954 - 29.95, 6)` da `0.004`.

El caso 6.996 (Parte B de la demostración): con pesos 249 y 1 y puntajes 7 y 6, el puntaje es (7 × 249 + 6 × 1) / 250 = 1749 / 250 = 6.996. Se muestra «7.00», pero el estado es «Puntaje insuficiente» frente a un mínimo de 7, porque la decisión se toma con el valor real. No es un error: la letra pide dos decimales para mostrar (pág. 4), no para decidir. El README del laboratorio lo explica.

6.5. Comparar reemplaza y los códigos no se reutilizan

En la Parte A de la demostración, la evaluación individual de Claude para el caso 1 recibe el código 1. Comparar el caso 1 la elimina y crea las 2, 3 y 4 (incluida una nueva de Claude). Comparar el caso 2 crea las 5, 6 y 7 y **conserva** las 2, 3 y 4. Comparar dos veces el mismo caso también funciona: si después del paso 18 se compara otra vez el caso 2, las 5, 6 y 7 desaparecen y se crean la 8, la 9 y la 10, con la fecha de ese momento; las 2, 3 y 4 del caso 1 no cambian (verificado ejecutando el programa; sección 8.4). Después de comparar, una evaluación individual de la misma opción y caso informa el duplicado.

6.6. Actualizar no cambia códigos ni fechas

- **Presupuesto del caso 1 a 300:** las evaluaciones siguen siendo la 2, 3 y 4, con las mismas fechas. Gemini pasa a Recomendada (300 = 300). El ranking del caso 1 queda: 1. Gemini (8.50, USD 300.00), 2. Claude (7.83, USD 200.00).

- **Precio de Claude a USD 70 por cada 1000 consultas, 5000 consultas:** costo nuevo 5000 / 1000 × 70 = USD 350.00. Claude queda «Fuera de presupuesto» en el caso 1 (evaluación 2) **y** en el caso 2 (evaluación 5); las demás evaluaciones de ambos casos conservan su código y su fecha: una actualización de precio recalcula en todos los casos. El puntaje sigue en 7.83 y el ranking del caso 1 queda solo con Gemini.

Si el caso no tiene evaluaciones, actualizar el presupuesto solo cambia el número (letra, pág. 4, Presupuesto de un caso).

6.7. Ranking y desempates

Solo entran las evaluaciones «Recomendada» del caso. Orden: mayor puntaje sin redondear; si empatan, menor costo; si persiste, menor código de opción.

- **Parte B, empate de puntaje:** Justa (USD 29.95) y Barata (USD 20.00) tienen ambas 7.0 exacto. El ranking da 1. Barata, 2. Justa: desempata el costo. Casi (6.996) no aparece porque no es Recomendada.
- **Empate de costo con ruido decimal:** el costo también se redondea a seis decimales en el ranking: 3 usuarios a USD 1.10 dan `3.3000000000000003` y 1 usuario a USD 3.30 da `3.3`. Sin redondeo, la segunda ganaría por una diferencia invisible; con redondeo empatan y decide el menor código de opción.

7. Decisiones de diseño y su porqué

Decisión	Alternativa descartada	Por qué
Solo lo que pide la letra	Agregar validaciones extra, pruebas automáticas, menús de borrar o editar	Código más fácil de explicar y corregir, con menos superficie de error
Dominio sin `input`/`print`	`input` dentro de `Sistema`, como en las diapositivas 10, 11 y 13	Respeta el diagrama, no repite la lectura y separa responsabilidades
Cada regla en una sola capa	Validar lo mismo en consola y dominio	Una regla escrita dos veces puede quedar distinta
Un único `__crear_evaluacion` para individual y comparar	Dos métodos que crean evaluaciones	La letra exige las mismas reglas en ambos (pág. 4)
Estado comparado con el puntaje sin redondear	Redondear antes de comparar	Redondear aceptaría 6.996 con mínimo 7
`round(costo - presupuesto, 6) > 0`	`costo > presupuesto`	5 × 5.99 no daría igual a 29.95
Contador de códigos por entidad	`len(lista) + 1`	Comparar elimina evaluaciones y `len` repetiría códigos
Una sola excepción propia	Varias subclases, o `ValueError`	La letra pide al menos una; cada mensaje ya distingue el error; `ValueError` se confundiría con el de `int()`
Atributos privados con `@property`	Atributos públicos o métodos `get_x()`	Nadie modifica un dato por accidente; solo cambia lo que la letra pide actualizar

Decisión	Alternativa descartada	Por qué
Nombre único sin mayúsculas también en opciones y casos	Distinguir mayúsculas en opciones y casos: la letra pide nombre único en los tres (pág. 3), pero aclara «sin distinguir mayúsculas» solo para criterios	Un único método sirve para las tres listas, y «Claude» y «CLAUDE» serían casi iguales en un listado
Nombre validado apenas se escribe	Validar al final	El usuario no completa datos en vano
Orden de registro de opción: nombre, tipo, costos, puntajes	Puntajes antes que costos (diapositiva 11)	Es el orden de la letra (pág. 3, Opción de IA); ante una diferencia, manda la letra
Sin `__init__.py`	Agregar archivos vacíos	La letra da la lista exacta de archivos y Python 3 importa las carpetas sin ellos
Sin librerías externas	Paquetes externos	`abc` y `datetime` alcanzan y no hay nada que declarar en el README

Otras cinco decisiones (objetos en lugar de códigos, `recalcular` en `Evaluacion`, `ranking_caso` en `Sistema`, puntaje y costo calculados bajo demanda, `match` en los menús) se responden en las preguntas 11 a 15 (sección 9.2).

Limitaciones aceptadas a propósito. Conviene conocerlas por si el profesor las encuentra:

- `leer_flotante` acepta `nan` e `inf`, porque `float("nan")` es un número válido para Python y `nan < 0` es falso.
- Una cantidad entera de cientos de dígitos se registra, pero el programa termina con `OverflowError` al calcular el costo.
- Ctrl+D (fin de entrada) o Ctrl+C durante una lectura terminan el programa con un error de Python.
- Las propiedades `criterios`, `opciones` y `casos` devuelven la lista interna, no una copia. `main.py` solo las lee.

Respuesta si aparecen: *«Es una limitación conocida y documentada; la corrección son dos líneas (rechazar `nan`/`inf` y poner un máximo a las cantidades), pero el equipo decidió no agregar código que la letra no pide.»*

8. Demostración en vivo

8.1. Preparación

1. Abrir una terminal en la carpeta `laboratorio_2026_grupo5/`.
2. Comprobar la versión: `python3 --version` debe ser 3.10 o mayor (con 3.9 el `match` da error de sintaxis).
3. Ejecutar `python3 main.py` (o `python main.py` si ese comando apunta a Python 3.10 o mayor).

Consejos: los decimales se escriben con **punto** (`7.5`, no `7,5`). Cada vez que hay que elegir una opción o un caso, el programa lista primero los registros con su código. Si algo se escribe mal, el programa avisa y vuelve a preguntar: no hace falta reiniciar. Si hubiera que reiniciar, se pierden los datos (no hay persistencia, letra, pág. 5).

8.2. Parte A: recorrido completo

Cada fila indica qué teclear, en orden, qué debe aparecer y en qué menú queda el programa. «Enter» después de cada valor. Cada menú imprime su título (`--- MENÚ PRINCIPAL ---`, `--- REGISTRAR ---`, `--- ACTUALIZAR ---`, `--- LISTAR ---`), así que siempre se puede ver en cuál se está.

Paso	Teclear	Debe aparecer	Menú al terminar
1	`abc`, `9`	«Ingrese un número entero.» y «Ingrese un número menor o igual que 5.»	Principal
2	`1` (Registrar), `1`, `Precisión`, `3`	«Criterio registrado con código 1.»	Registrar
3	`1`, `Velocidad`, `2`	código 2	Registrar
4	`1`, `precisión`	«Error: Ya existe un registro llamado precisión.» (no pide el peso)	Registrar
	`Soporte`, `1`, y `5` (Salir)	código 3; vuelve al menú principal	Principal
5	`4` (Listar), `1`, y `6`	`Código: 1 | Nombre: Precisión | Peso: 3`, y Velocidad 2, Soporte 1	Principal
6	`1`, `2`, `Claude`, `1`, `40`, `5000`, `8`, `7`, `9`	«Opción registrada con código 1.»	Registrar
7	`2`, `OpenAI`, `2`, `10`, `15`, `6`, `7`, `5`	código 2	Registrar
8	`2`, `Gemini`, `1`, `30`, `10000`, `9`, `8`, `8`, y `5`	código 3	Principal
9	`4`, `2`, y `6`	costos USD 200.00, 150.00 y 300.00, con sus puntajes	Principal
10	`1`, `1`	«Ya hay opciones registradas: no se admiten nuevos criterios.»	Registrar
11	`3`, `Atención al cliente`, `250`, `7`	«Caso registrado con código 1.»	Registrar
12	`3`, `Análisis de documentos`, `300`, `7.5`, y `5`	código 2	Principal
13	`4`, `3`, y `6`	`Recomendadas: 0` en ambos casos	Principal
14	`1`, `4`, `1` (opción), `1` (caso)	`Código: 1 | ... | Puntaje: 7.83 | Costo: USD 200.00 | Estado: Recomendada`	Registrar
15	`4`, `1`, `1`, y `5`	«Error: Ya existe una evaluación de Claude para Atención al cliente.»	Principal
16	`2` (Comparar), `1`	códigos 2, 3 y 4: Claude 7.83 Recomendada; OpenAI 6.17 Puntaje insuficiente; Gemini 8.50 Fuera de presupuesto	Principal
17	`4`, `3`, y `6`	caso 1: `Recomendadas: 1`; caso 2: `Recomendadas: 0`	Principal
18	`2`, `2`	códigos 5, 6 y 7: Recomendada, Puntaje insuficiente, Recomendada	Principal
19	`4`, `4`, `1`, y `6`	siguen los códigos 2, 3 y 4, con `Presupuesto: USD 250.00`; anotar la fecha	Principal
20	`3` (Actualizar), `1`, `1`, `300`, y `3`	«Presupuesto de Atención al cliente actualizado a USD 300.00.»	Principal
21	`4`, `4`, `1`	`Presupuesto: USD 300.00`; Gemini pasa a Recomendada; mismos códigos y fechas	Listar
22	`5`, `1`, y `6`	`1. Gemini | Puntaje: 8.50 | Costo: USD 300.00` y `2. Claude | Puntaje: 7.83 | Costo: USD 200.00`	Principal
23	`3`, `2`, `1`, `70`, `5000`, y `3`	«Opción Claude actualizada: costo mensual USD 350.00.»	Principal
24	`4`, `2`	Claude con USD 350.00	Listar
25	`4`, `1`	caso 1: Claude USD 350.00, 7.83, Fuera de presupuesto; códigos 2, 3, 4	Listar

Paso	Teclear	Debe aparecer	Menú al terminar
26	4, 2	caso 2: Claude USD 350.00, 7.83, Fuera de presupuesto; códigos 5, 6, 7	Listar
27	5, 1, y 6	solo `1. Gemini | Puntaje: 8.50 | Costo: USD 300.00`	Principal
28	5	«Programa finalizado.»	(fin)

Qué decir en cada momento clave:

- **Paso 4:** «La unicidad no distingue mayúsculas y se valida antes de pedir el peso.»
- **Paso 10:** «Una vez que hay opciones, los criterios se cierran; si no, una opción quedaría sin puntaje para el criterio nuevo.»
- **Paso 16:** «Comparar reemplazó la evaluación 1; los códigos siguen desde 2. Gemini tiene el mejor puntaje pero queda fuera porque primero se mira el presupuesto.»
- **Paso 19:** «Comparar el caso 2 no tocó las evaluaciones del caso 1.»
- **Paso 21:** «Costo igual al presupuesto es válido. No se crearon evaluaciones ni cambiaron las fechas.»
- **Pasos 25 y 26:** «Cambiar el precio recalcula en todos los casos; el puntaje no cambia.»

8.3. Parte B: casos borde

Reiniciar el programa.

Paso	Teclear	Debe aparecer	Menú al terminar
1	1, 1, Exactitud, 249	código 1	Registrar
2	1, Rapidez, 1	código 2	Registrar
3	2, Justa, 2, 5, 5.99, 7, 7	código 1	Registrar
4	2, Casi, 2, 1, 10, 7, 6	código 2	Registrar
5	2, Barata, 2, 1, 20, 7, 7	código 3	Registrar
6	3, Bordes, 29.95, 7, y 5	caso código 1	Principal
7	2, 1	Justa: USD 29.95, 7.00, Recomendada. Casi: USD 10.00, 7.00, Puntaje insuficiente. Barata: USD 20.00, 7.00, Recomendada	Principal
8	4, 5, 1, y 6	`1. Barata | Puntaje: 7.00 | Costo: USD 20.00` y `2. Justa | Puntaje: 7.00 | Costo: USD 29.95`	Principal
9	5	«Programa finalizado.»	(fin)

Qué decir: «Justa cuesta exactamente el presupuesto y tiene exactamente el mínimo: es válida. Casi muestra 7.00 pero su puntaje real es 6.996, menor que 7. En el ranking, Barata y Justa empatan en puntaje y gana la de menor costo.»

8.4. Pedidos improvisados

Si el profesor pide algo fuera del guion, se parte del menú principal (salir de un submenú: 5 en Registrar, 3 en Actualizar, 6 en Listar). Todas las respuestas se comprobaron ejecutando el programa.

Pedido del profesor	Teclear desde el menú principal	Qué debe aparecer
Comparar otra vez el caso 2 (después del paso 18)	`2`, `2`	Códigos 8, 9 y 10; las 5, 6 y 7 desaparecen; las 2, 3 y 4 del caso 1 siguen
Evaluaciones de un caso sin evaluaciones (antes del paso 14)	`4`, `4`, `1`, `6`	«No hay evaluaciones para Atención al cliente.»
Ranking de un caso sin recomendadas (antes del paso 18)	`4`, `5`, `2`, `6`	«No hay evaluaciones recomendadas para Análisis de documentos.»
Comparar con el programa recién iniciado	`2`	«No hay opciones registradas para comparar.»
Registrar una opción sin criterios (recién iniciado)	`1`, `2`, `5`	«Debe existir al menos un criterio para registrar opciones.»
Evaluación individual sin casos (recién iniciado)	`1`, `4`, `5`	«No hay casos de uso registrados para seleccionar.»
Registrar un criterio con opciones ya registradas	`1`, `1`, `5`	«Ya hay opciones registradas: no se admiten nuevos criterios.»

Si se compara otra vez el caso 2 en medio de la Parte A, el paso 26 muestra los códigos 8, 9 y 10 en lugar de 5, 6 y 7; los pasos 19, 21 y 25 (caso 1) siguen mostrando 2, 3 y 4, y el resto del guion no cambia.

9. Preguntas probables del profesor

9.1. Conceptos

1. ¿Dónde está el polimorfismo? En `Evaluacion.costo_total()`, que llama a `self.__opcion.costo_estimado()` sin saber si la opción es `ServicioAPI` o `PlanMensual`. Cada clase responde con su fórmula. También en `tipo`, `descripcion_costos()` y `actualizar_costos()`, aunque esos tres no están declarados en `OpcionIA`: funcionan porque las dos derivadas los escriben con el mismo nombre (sección 4.2).

2. ¿Por qué `OpcionIA` es abstracta? ¿Qué pasa si intento crear una? Porque una opción sin tipo no tiene fórmula de costo. `ABC` y `@abstractmethod` impiden crearla (`TypeError`, sección 3.1) y obligan a cada derivada a implementar `costo_estimado()`, que es lo que la letra exige (pág. 3, Opción de IA).

3. ¿Para qué sirve `super().__init__`? Para que el constructor del padre cargue `codigo`, `nombre` y `puntajes`. La derivada no repite ese código y no puede acceder a `self.__codigo` del padre por el *name mangling* (sección 3.4); usa la propiedad `self.codigo`.

4. ¿Los atributos con `__` son realmente privados? Python los renombra (`__peso` pasa a ser `_Criterio__peso`), así que desde afuera `criterio.__peso` falla. Es una protección contra el uso accidental, no un candado: con el nombre interno se puede acceder.

5. ¿Por qué hay un solo setter? Porque solo el presupuesto se actualiza como un dato suelto (letra, pág. 4). Los costos de una opción cambian juntos, en pares, y usan el método `actualizar_costos`. Lo demás nunca cambia.

6. ¿Por qué `ErrorLaboratorio` y no `ValueError`? La letra pide al menos una excepción propia (pág. 2). Además, `ValueError` es lo que lanza `int("abc")`, y se mezclarían dos errores distintos.

7. ¿Por qué no capturan `Exception`? Porque escondería errores de programación como si fueran del usuario. Se capturan solo `ValueError` (al convertir) y `ErrorLaboratorio` (reglas).

8. ¿Por qué `Evaluacion` guarda el objeto y no el código? Lo pide la letra (pág. 2) y es más útil: la evaluación lee el precio actual de la opción y el presupuesto actual del caso sin buscar nada. Si guardara el código, cada cálculo tendría que buscar la opción en `Sistema`, que `Evaluacion` no conoce. Para saber si dos evaluaciones son de la misma opción se compara el código: dentro de un `Sistema` cada código identifica a un único objeto y nunca cambia, así que comparar códigos equivale a comparar objetos y se lee igual que en los listados.

9. ¿Por qué el diccionario usa el código del criterio como clave? Lo fija la letra (pág. 2). El código es único e inmutable, y el diccionario impide dos puntajes para el mismo criterio.

9.2. Diseño

10. ¿Por qué no validan el nombre vacío en `Sistema`? Porque cada regla se valida en una sola capa. «No vacío» se decide mirando solo el texto, así que lo valida la consola (`leer_texto`). «No repetido» exige mirar lo registrado, así que lo valida `Sistema.validar_nombre`. Consecuencia honesta: si alguien llamara a `Sistema` sin pasar por `main.py`, podría registrar un nombre vacío (verificado); el diseño confía en que el dominio recibe datos ya validados.

11. ¿Por qué los métodos de `Sistema` reciben objetos y no códigos? Porque `main.py` ya obtuvo el objeto al seleccionarlo con `buscar`. Recibir el código obligaría a buscar dos veces.

12. ¿Por qué `recalcular` está en `Evaluacion`? Porque es la clase que tiene la opción, el caso y el estado. `Sistema.recalcular` solo recorre las evaluaciones afectadas y le pide a cada una que se recalcule.

13. ¿Por qué `ranking_caso` está en `Sistema`? Porque necesita todas las evaluaciones del caso y la lista de criterios, y ambas son de `Sistema`. `main.py` solo lo muestra.

14. La letra dice que la evaluación «registra puntaje y costo» (pág. 2, Clases). ¿Por qué no los guardan? La evaluación los ofrece siempre que se le piden (`puntaje_total()` y `costo_total()`), igual que en el diagrama (sección 2.5), donde figuran como métodos. Guardar el costo lo dejaría desactualizado al cambiar el precio. Solo se guarda el estado, y por eso existe `recalcular`.

15. ¿Por qué usan `match`/`case` y no `if`/`elif` como las diapositivas? Fue una elección del equipo: la letra exige Python 3.10 o mayor, donde `match` existe, y para un menú de valores fijos se lee como una lista vertical. Es equivalente al `if`/`elif` de las diapositivas.

9.3. Reglas de la letra

16. ¿Por qué `round(costo - presupuesto, 6)`? Porque `5 * 5.99` da `29.950000000000003`. Sin redondear, un costo que en pantalla es igual al presupuesto quedaría «Fuera de presupuesto», y la letra dice que la igualdad es válida (pág. 4). ¿Por qué seis y no dos? Con dos, 29.954 frente a 29.95 daría diferencia 0.00 y se aceptaría un costo que supera el presupuesto; seis eliminan el ruido del `float` y detectan diferencias reales mucho menores que un centavo (sección 6.4).

17. ¿Por qué se muestra 7.00 con «Puntaje insuficiente»? Porque el puntaje real es 6.996 y el estado se decide con el valor real. La letra pide dos decimales solo para mostrar.

18. ¿En qué orden se decide el estado? Primero presupuesto, después puntaje (letra, pág. 4). Una opción cara y mala es «Fuera de presupuesto».

19. ¿Cómo se ordena el ranking? Solo recomendadas; mayor puntaje sin redondear; empate por menor costo; después por menor código de opción. Se implementa con `sorted` y una tupla como clave.

20. ¿Por qué no se pueden agregar criterios después de la primera opción? Lo exige la letra (pág. 3, Criterio). Además, las opciones existentes no tendrían puntaje para el criterio nuevo y `puntaje_total` fallaría al buscarlo en el diccionario.

9.4. ¿Qué pasa si…?

21. …comparo dos veces el mismo caso? Se eliminan las evaluaciones anteriores del caso y se crean nuevas con códigos nuevos y fecha nueva. No se duplican y los códigos viejos no se reutilizan (se demuestra en la sección 8.4).

22. …hago una evaluación individual después de comparar? Informa el duplicado, porque la comparación ya creó una evaluación para esa opción y ese caso.

23. …comparo un caso que tenía evaluaciones individuales? Se reemplazan también (letra, pág. 4). En la demostración, la evaluación 1 desaparece.

24. …comparo sin opciones registradas? Muestra «No hay opciones registradas para comparar.» y vuelve al menú, sin pedir el caso.

25. …actualizo el presupuesto de un caso sin evaluaciones? Solo cambia el presupuesto (letra, pág. 4).

26. …escribo el nombre con espacios alrededor, como « Claude »? `strip()` los quita; si ya existe «Claude», se informa el duplicado.

27. …ingreso un código que no existe? «Error: No existe un registro con código N.» y vuelve a pedir el código.

28. …el peso es 0 o negativo? Se rechaza en `leer_entero(..., 1)`. Por eso la suma de pesos nunca es cero y `puntaje_total` nunca divide por cero.

29. …quiero agregar un tercer tipo de opción? Se crea una clase derivada de `OpcionIA` con su `costo_estimado()`, `tipo`, `descripcion_costos()` y `actualizar_costos()`; Python solo obliga a escribir el primero (sección 4.2), así que los otros tres hay que acordarse de agregarlos. `Evaluacion` y `Sistema.ranking_caso` no cambian. En `main.py` habría que agregar las preguntas de ese tipo en `registrar_opcion` y `actualizar_precio`, y un `registrar_*` en `Sistema`.

10. Errores que no hay que cometer y glosario

10.1. Afirmaciones falsas frecuentes

No decir	Lo correcto
«La evaluación guarda el puntaje y el costo.»	Solo guarda el estado; puntaje y costo se calculan al pedirlos.
«El estado se decide con el puntaje redondeado.»	Se decide con el valor real; el redondeo es solo para mostrar.
«El `round(..., 6)` redondea el costo que se muestra.»	Ningún `round(..., 6)` cambia lo que se muestra (eso usa `:.2f`). En `Evaluacion.recalcular` redondea la diferencia costo − presupuesto para decidir el estado; en `Sistema.ranking_caso` redondea el costo para desempatar el ranking.
«Comparar agrega evaluaciones nuevas.»	Reemplaza todas las del caso; las de otros casos se conservan.

No decir	Lo correcto
«El código es la cantidad de elementos más uno.»	Es un contador por entidad que nunca baja.
«Actualizar el presupuesto vuelve a comparar.»	Solo recalcula estados; no crea evaluaciones ni cambia fechas.
«`Sistema` valida que el nombre no esté vacío.»	Lo valida `leer_texto` en `main.py`; `Sistema` valida duplicados.
«`Sistema` muestra los errores.»	`Sistema` lanza `ErrorLaboratorio`; `main.py` lo captura y lo muestra.
«`main.py` captura cualquier error.»	Solo `ValueError` y `ErrorLaboratorio`.
«`OpcionIA.costo_estimado()` devuelve 0 por defecto.»	Es abstracto: en este programa nunca se ejecuta y `OpcionIA` no se puede instanciar.
«El ranking ordena todas las evaluaciones.»	Solo las «Recomendada».
«Los atributos privados no se pueden leer de ninguna forma.»	Python los renombra; es protección contra accidentes, no un candado.
«Los pesos tienen que sumar 100.»	La letra dice expresamente que no (pág. 3, Criterio).
«El puntaje mínimo es entero.»	Es un número entre 0 y 10 y admite decimales (por ejemplo, 7.5). Los enteros son los puntajes por criterio y el peso.
«La fecha se toma en `main.py`.»	Se toma en `Sistema.__crear_evaluacion` con `datetime.now()`.

10.2. Glosario

- **Clase abstracta:** clase que no se puede instanciar y define métodos que las derivadas deben implementar (`ABC`, `@abstractmethod`).
- **Herencia:** una clase derivada reutiliza lo de su clase padre y agrega lo propio (`class ServicioAPI(OpcionIA)`).
- **Polimorfismo:** el mismo mensaje (`costo_estimado()`) produce resultados distintos según el objeto que lo recibe.
- **Encapsulamiento:** ocultar los datos de un objeto y ofrecer un acceso controlado (`__atributo`, `@property`).
- **Propiedad:** método que se usa como atributo, sin paréntesis. Con `@x.setter` también permite asignar.
- **Name mangling:** Python renombra `__x` dentro de la clase `C` a `_C__x`.
- **Decorador:** línea con `@` sobre una función que le agrega un comportamiento (`@property`, `@abstractmethod`).
- **Excepción:** objeto que interrumpe la ejecución con `raise` y se maneja con `try/except`.
- **Referencia:** variable que apunta a un objeto existente, no a una copia.
- **`float`:** número decimal guardado en binario; algunas cuentas dejan restos mínimos (`29.950000000000003`).
- **f-string:** texto con valores insertados, como `f"{puntaje:.2f}"`, que muestra dos decimales.
- **`match`/`case`:** selección por valor, disponible desde Python 3.10.
- **Docstring:** texto entre comillas triples bajo una función o clase que documenta qué hace.
- **Tupla como clave de orden:** `sorted` compara el primer elemento y, si empatan, el siguiente.
- **`lambda`:** función anónima de una sola expresión; se usa para pasar la clave de orden a `sorted`.